



TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

Scaling neural operators for large-scale plasma simulations and coupling to production codes

October 2026 | Nikola Mamić | JSC



INTRODUCTION

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

SIMULATING PLASMAS

FIGURE: Phase space

$f(\mathbf{x}, \mathbf{v})$ as colour map,
macro-particles as dots

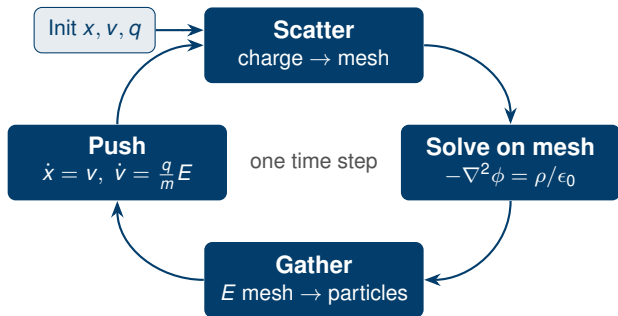
[figures/phase_space.pdf](#) | [png](#)

particles sample the distribution $f(\mathbf{x}, \mathbf{v})$

- **Plasma**: ionised gas, electrons and ions coupled by electromagnetic fields; fusion, particle accelerators, space physics
- Described by a **distribution function** $f(\mathbf{x}, \mathbf{v}, t)$: 6 dimensions + time, evolving by the Vlasov equation
$$\partial_t f + \mathbf{v} \cdot \nabla_{\mathbf{x}} f + \frac{q}{m} (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{v}} f = 0$$
- A 6D grid is unaffordable: 100 points per dimension $\rightarrow 10^{12}$ cells
- **Particles** sample f , and f is constant along particle paths $\rightarrow$ move particles by **ODEs**; only the **field** needs a grid (3D)

PARTICLE–FIELD SYSTEMS AND THE PIC LOOP

- **Particles** move by ODEs, **fields** obey PDEs
- Coupled in both directions: particles make the field, the field pushes the particles
- Plasmas (particle-in-cell, PIC), vortex flows, ...
- Electrostatic plasma:
 $-\nabla^2 \phi = \rho / \epsilon_0, \quad E = -\nabla \phi$



FROM GRIDS TO FOURIER MODES

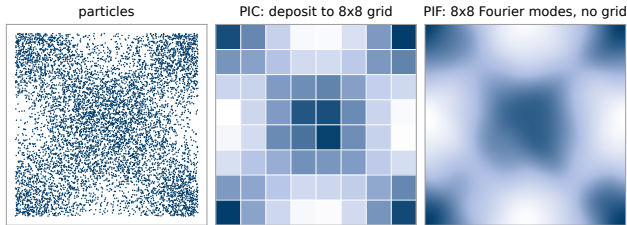
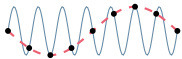


Illustration with synthetic particles, both with 64 degrees of freedom

PIC: the grid causes trouble



aliasing → grid
heating, energy drift



cells must resolve
the **Debye length**

PIF: particles talk to Fourier modes

- $\hat{\rho}_k = S_k \sum_j q_j e^{-ik \cdot x_j}$ - no real-space grid
- No grid aliasing, spectral accuracy
- With NUFFT: same cost order as PIC, higher constant
- Limits: global operation; periodic / free-space boundaries

WHY LEARN THE FIELD OPERATOR?

$\sim 10^{11}$
grid points / modes

$\sim 10^{12}$
particles

$\sim 10^5$
time steps

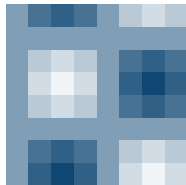
for one high-fidelity run (fusion, particle accelerators)

Motivation for surrogates:

- PIC and PIF **do** scale well on today's supercomputers, but...
- High-fidelity and many-query studies remain **too expensive**
- Implicit / larger time steps via autograd Jacobians
- Differentiable simulations
- Field operators that are expensive or not known exactly
- One model for many codes

DEEP LEARNING SURROGATES

- **Foundation models**: pretrain once, fine-tune for new tasks
- **Function to function**: not tied to one grid or resolution"
- Precedent - **Poseidon** (Herde et al., NeurIPS 2024): pretrained on a **small** set of fluid PDEs (compressible Euler, incompressible Navier–Stokes), generalises to unseen PDEs
- **Differentiable** end to end: gradients for optimisation, inverse problems, implicit time stepping

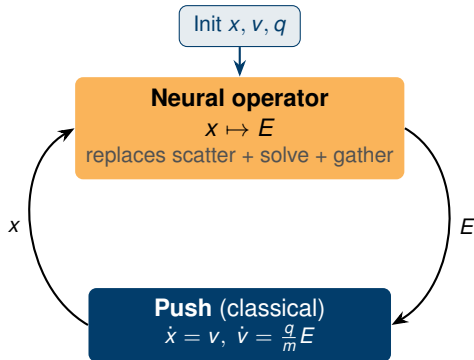


fields on a grid
(Poseidon)



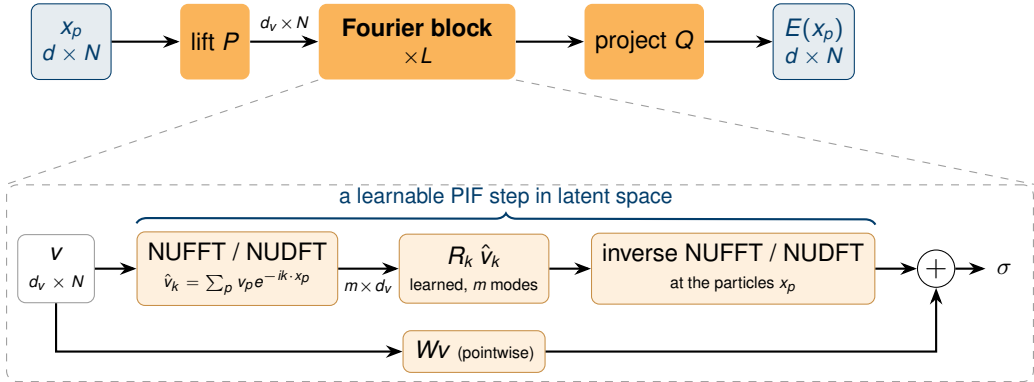
particles anywhere
(our data)

THE PROJECT'S APPROACH



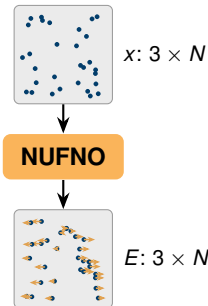
- Learn the whole map $P^T \mathcal{L}^{-1} P : x \mapsto E$
- Particle push stays classical
- Training data from mini-apps (PIF/PIC)
- Input: particle positions $\rightarrow$ output: E at the particles

THE MODEL: NONUNIFORM FNO



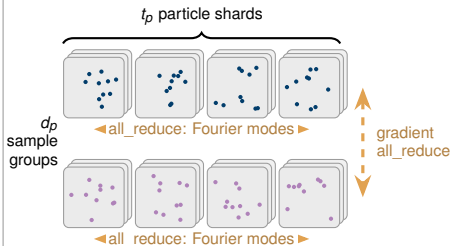
WHAT GOES IN, WHAT COMES OUT

One sample



input: particle positions
output: E at each particle

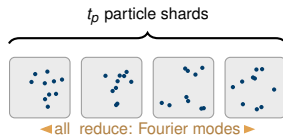
Training



each cell = one GPU, holding
 b samples $\times$ n particles

$$W = d_p t_p \quad b = B/d_p \quad n = N/t_p$$

Inference



$B = 1$: one configuration
per time step

$$d_p = 1, \quad W = t_p$$



THE PROJECT AND THE CONTRIBUTION

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

THE PROJECT

- NUFNO model
- Distributed training: data parallel (d_p) $\times$ particle decomposition (t_p)
- t_p (tensor parallel in the code) = particle decomposition
- picND: evaluation against a PIF reference
- Training data from the mini-apps

THE PROJECT

- NUFNO model
- Distributed training: data parallel (d_p) $\times$ particle decomposition (t_p)
- t_p (tensor parallel in the code) = particle decomposition
- picND: evaluation against a PIF reference
- Training data from the mini-apps

The contributions made:

Scaling inference out

from one GPU holding all particles to many GPUs

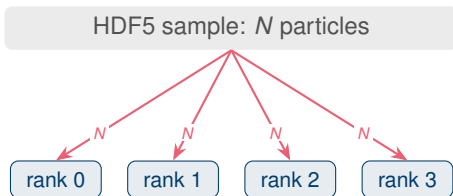
Leaner I/O

each rank reads only the particles it needs

Into IPPL

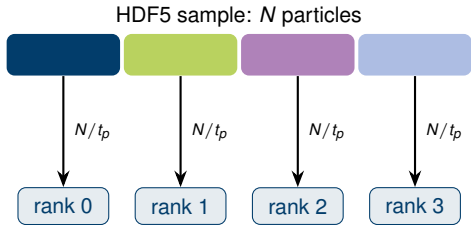
run the model as a surrogate inside a simulation library

TRAINING I/O: SHARDED HDF5 READS



before: each rank reads N , keeps N/t_p

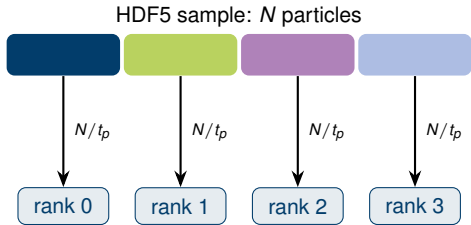
TRAINING I/O: SHARDED HDF5 READS



after: each rank reads only its shard

- Read volume per rank: $N \rightarrow N/t_p$

TRAINING I/O: SHARDED HDF5 READS



after: each rank reads only its shard

- Read volume per rank: $N \rightarrow N/t_p$

2.17 TiB

$t_p = 1$

4.34 TiB

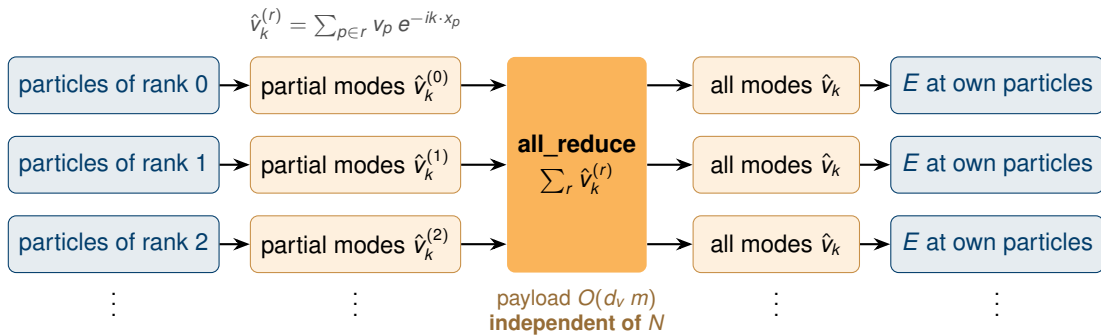
$t_p = 2$, before fix

2.15 TiB

$t_p = 2$, after fix

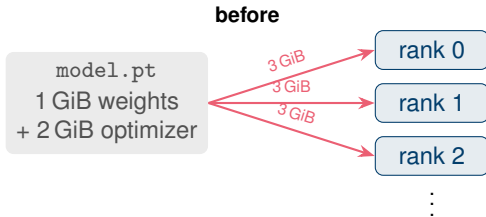
total data read from disk, 100 epoch runs

PARALLEL INFERENCE



- Particle decomposition in `picND`: each GPU holds N/W particles; in inference $W = t_p$ (no data parallelism)

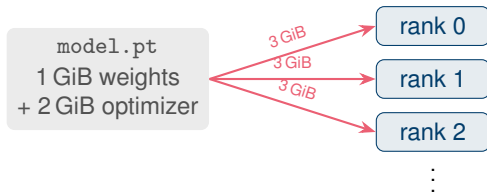
INFERENCE I/O: READ ONCE, BROADCAST



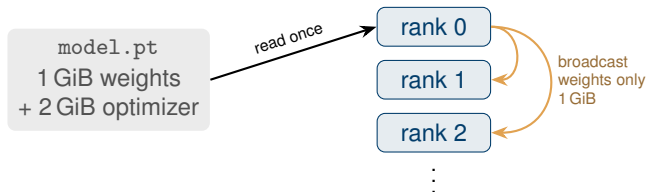
- Checkpoint: **1 GiB** model weights + **2 GiB** optimizer state, inference only needs weights
- Before: **every rank** reads whole file: $W \times 3 \text{ GiB}$ from the shared file system

INFERENCE I/O: READ ONCE, BROADCAST

before



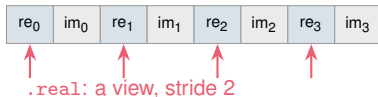
after



- Checkpoint: **1 GiB** model weights + **2 GiB** optimizer state, inference only needs weights
- Before: **every rank** reads whole file: $W \times 3$ GiB from the shared file system
- After: **rank 0** reads once, keeps only weights, broadcasts over the communicator
- Disk reads no longer grow with the GPU count

ENFORCING CONTIGUITY

complex tensor in memory



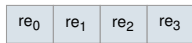
- `torch.permute()` and `torch.real` used at various points in the model
- return **views**: same memory, strided access
- Following kernels work on non-contiguous memory

ENFORCING CONTIGUITY

complex tensor in memory



↑
↑
↑
↑
`.real: a view, stride 2`



`.contiguous()`: one copy, dense

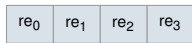
- `torch.permute()` and `torch.real` used at various points in the model
- return **views**: same memory, strided access
- Following kernels work on non-contiguous memory
- `.contiguous()` at the right places: dense memory for all following kernels

ENFORCING CONTIGUITY

complex tensor in memory



↑
↑
↑
↑
`.real: a view, stride 2`



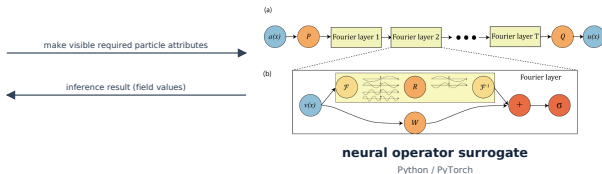
`.contiguous(): one copy, dense`

- `torch.permute()` and `torch.real` used at various points in the model
- return **views**: same memory, strided access
- Following kernels work on non-contiguous memory
- `.contiguous()` at the right places: dense memory for all following kernels



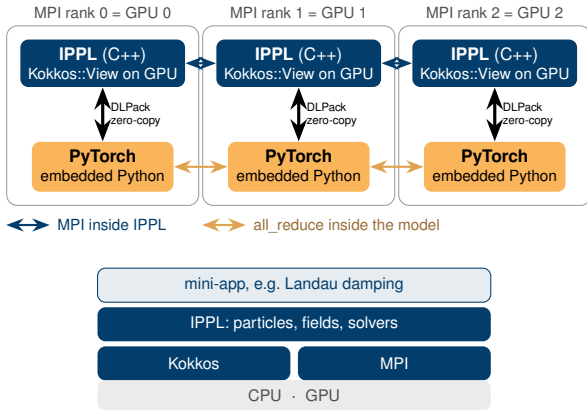
per time step, $\approx 1.27 \times$ faster

THE IPPL LIBRARY



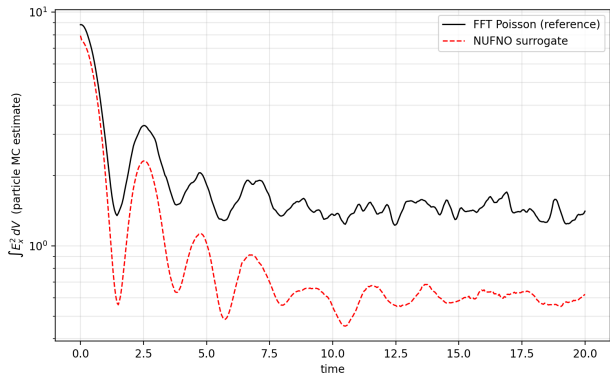
- IPPL: C++
particle-mesh library
- Performance portable:
Kokkos (CPU and GPU) + **MPI**
- Underlies the production
accelerator code
OPAL-X

COUPLING TO IPPL



- Time step unchanged: only the **field evaluation** is swapped for the model
- Same MPI rank hosts IPPL and the model: **rank i talks to rank i**
- Data stays on the GPU; Python writes E **directly into IPPL's memory**

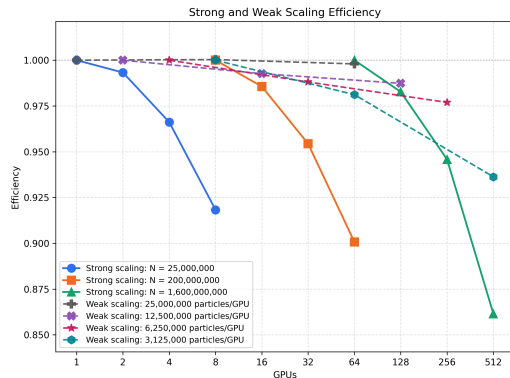
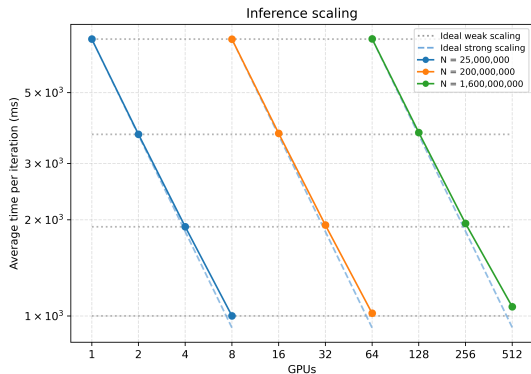
COUPLING TO IPPL



IPPL numeric solver vs. the coupled model surrogate

SCALING RESULTS

NUFFT model, run **through the IPPL interop layer**: 400 time steps, 1–512 GPUs, up to 1.6 B particles



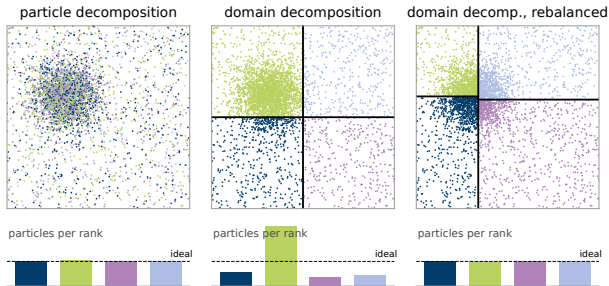
efficiency: strong $E_s = P_0 T(P_0) / (P T(P))$, weak $E_w = T_{\text{base}} / T(P)$



FINER DETAILS

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

TWO WAYS TO SPLIT THE PARTICLES



Particle decomposition (The project)

each rank: random subset over the **whole** domain, always balanced

Domain decomposition (IPPL)

each rank: **one subdomain**, needs re-partitioning → subdomain boundaries move

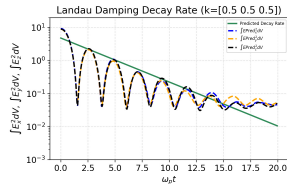
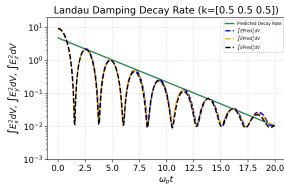
Under domain decomposition, rebalancing needed **per timestep** due to particle movement.

THE RISING NOISE FLOOR

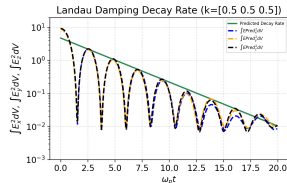
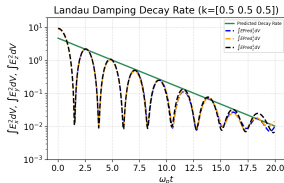
```
# before: same seed on every rank  
cp.random.seed(152)
```

```
# fix: different seed per rank  
cp.random.seed(152 + 100*rank)
```

- Every GPU draws its N/W particles from the **same seed** → very similar particles on all ranks
- Effectively only $\approx N/W$ independent particles → noise floor **grows with the GPU count**
- One seed per rank → the noise floor no longer depends on W

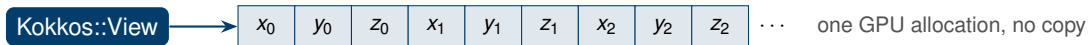


1 GPU vs. 8 GPUs, **before** (same seed)

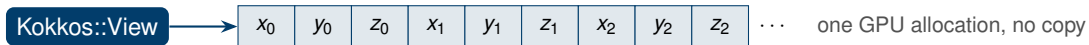


1 vs. 8 GPUs, **after** (seed + 100·rank)

ZERO-COPY VIA DLPACK



ZERO-COPY VIA DLPACK



DLTensor{ data, device=CUDA,
dtype=float64, shape=(N,3),
strides=(3,1) } + deleter

Zero-copy only if standard layout, no padding:
`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

ZERO-COPY VIA DLPACK

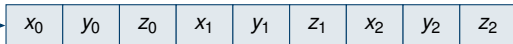
ParticleAttrib<Vector<double,3>

Kokkos::View<Vector<double,3>*>

shape (N, 3) of double

IPPL AoS → scalars

Kokkos::View



one GPU allocation, no copy

```
DLTensor{ data, device=CUDA,  
dtype=float64, shape=(N,3),  
strides=(3,1) } + deleter
```

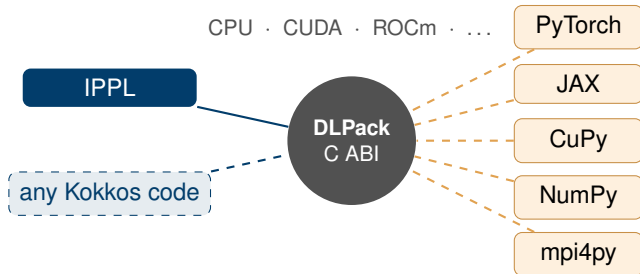
Zero-copy only if standard layout, no padding:
`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

Lifetime

the capsule keeps the Kokkos View alive until PyTorch releases it

ZERO-COPY VIA DLPACK

Why it matters beyond this project



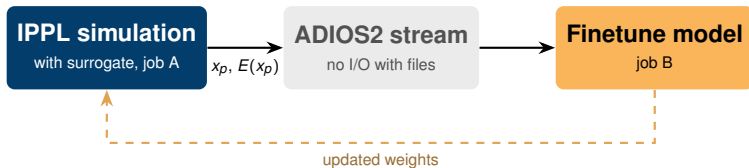
- Open standard: NumPy, CuPy, PyTorch, TensorFlow, JAX, MXNet, TVM, Paddle, mpi4py
- Approach usable for general simulation code
- Export side is generic over Kokkos Views, model side not tied to PyTorch



OUTLOOK

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

STREAMING DATA FOR ONLINE FINE-TUNING



- **Loose coupling**: simulation and trainer run as separate jobs; data is streamed, not written to disk
- **Online fine-tuning**: the model keeps learning from the running simulation, e.g. in new parameter regimes
- Utilizes easy switching between surrogate and numerical solver in simulation

ACKNOWLEDGEMENTS

- **Dr. Sriramkrishnan Muralikrishnan** – supervisor
- **Chelsea Maria John** – co-supervisor

Thank you!

Questions?